Analysis Framework

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Not everything that can be counted counts, and not everything that counts can be counted.

—Albert Einstein (1879–1955)

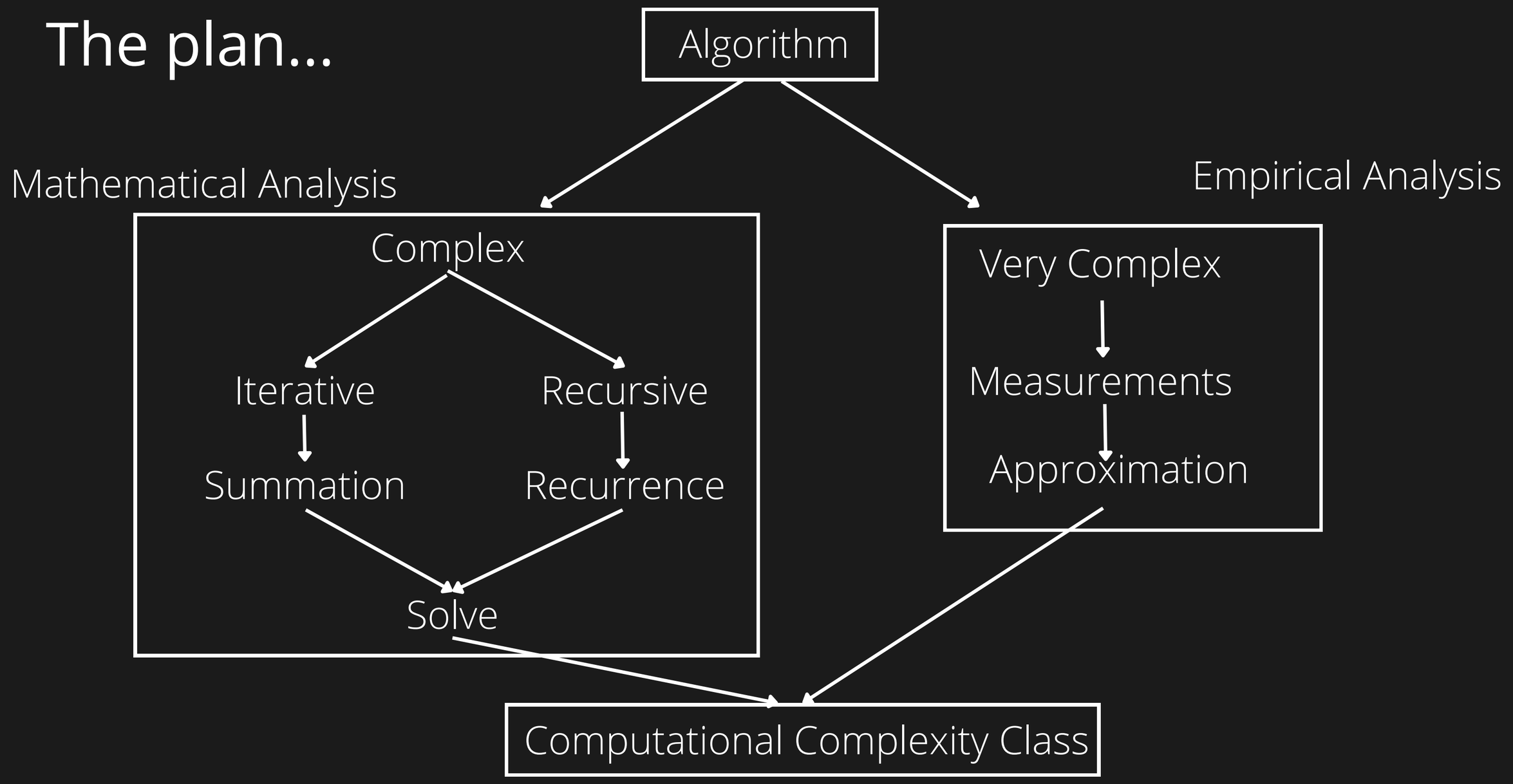
Issues:

- Correctness
- Time efficiency
- Space efficiency
- Optimality
- Time execution

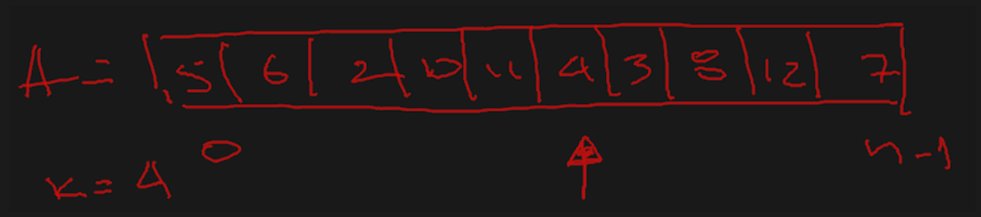
Approaches:

- Mathematical analysis
- Empirical analysis

The plan...


Algorithm
Mathematical Analysis
Empirical Analysis
Complex
Iterative
Recursive
Summation
Recurrence
Solve
Very Complex
Measurements
Approximation
Computational Complexity Class

Basic Operation




```
ALGORITHM SequentialSearch(A[0..n-1], K)
    // Search for a given value in a given array by sequential search
    // Input: An array A[0..n-1] and search key K
    // Output: The index of the first element A that matches K
    //         or -1 if there are no matching elements
    i ← 0
    while i < n and A[i] ≠ K do
        i ← i + 1
    if i < n return i
    else return -1
```

← Basic Operation

Basic Operation

Problem	*Input size measure*	*Basic operation*
Searching for key in a list of *n* items	Number of list's items, i.e. *n*	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer *n*	*n*'size = number of digits (in binary representation)	Division
Typical graph problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Order of Growth

TABLE 2.1 Values (some approximate) of several functions important for analysis of algorithms

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Worst-Case, Best-Case, Average-Case

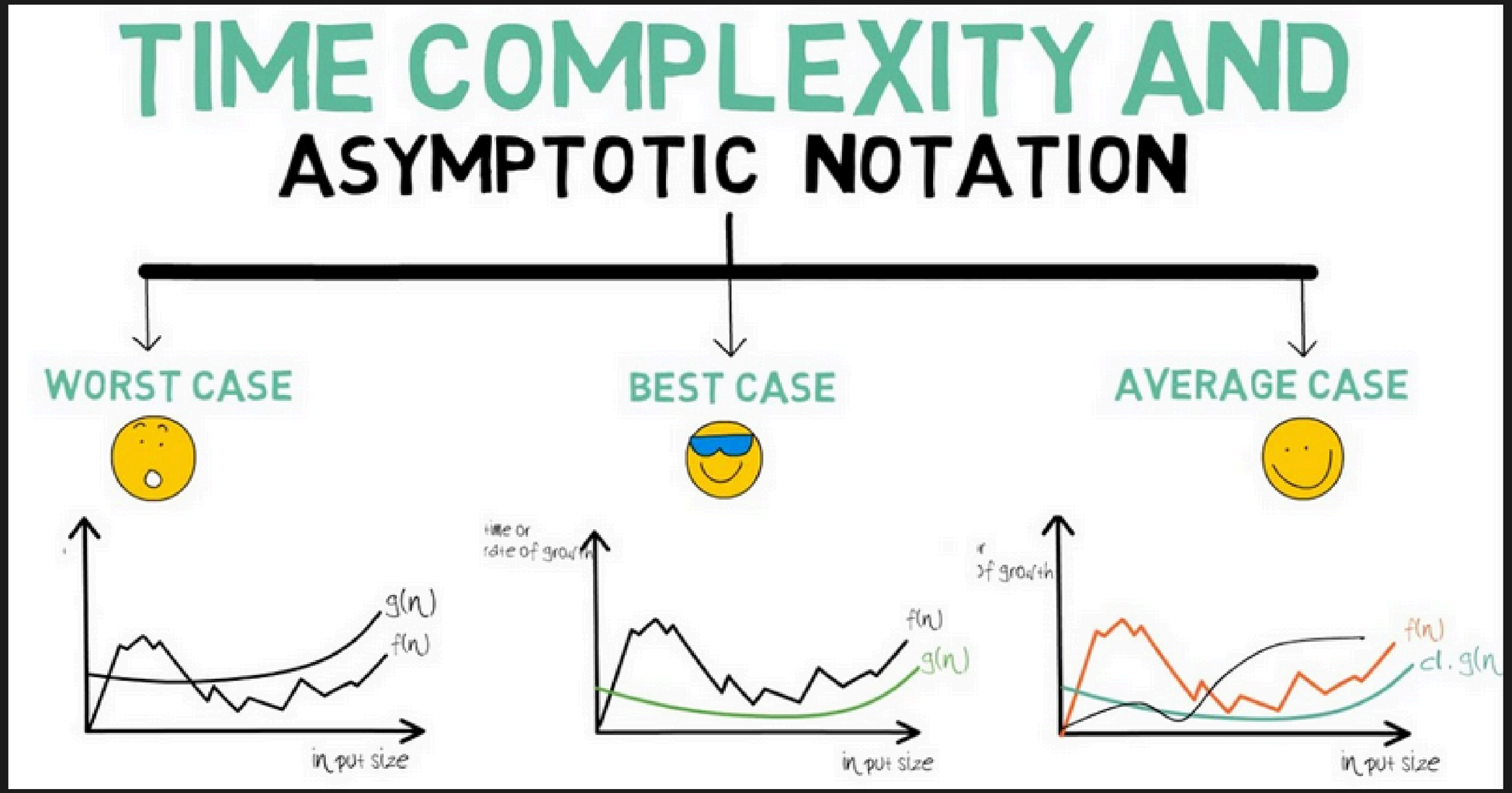
```
ALGORITHM SequentialSearch(A[0..n-1], K)
    // Search for a given value in a given array by sequential search
    // Input: An array A[0..n-1] and search key K
    // Output: The index of the first element A that matches K
    //         or -1 if there are no matching elements
    i ← 0
    while i < n and A[i] ≠ K do
        i ← i + 1
    if i < n return i
    else return -1
```

$O(n)$

$\Omega(1)$

$\Theta(\quad)$

Asymptotic Notations



O-notation

DEFINITION A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n, i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$t(n) \leq cg(n) \quad \text{for all } n \geq n_0.$$

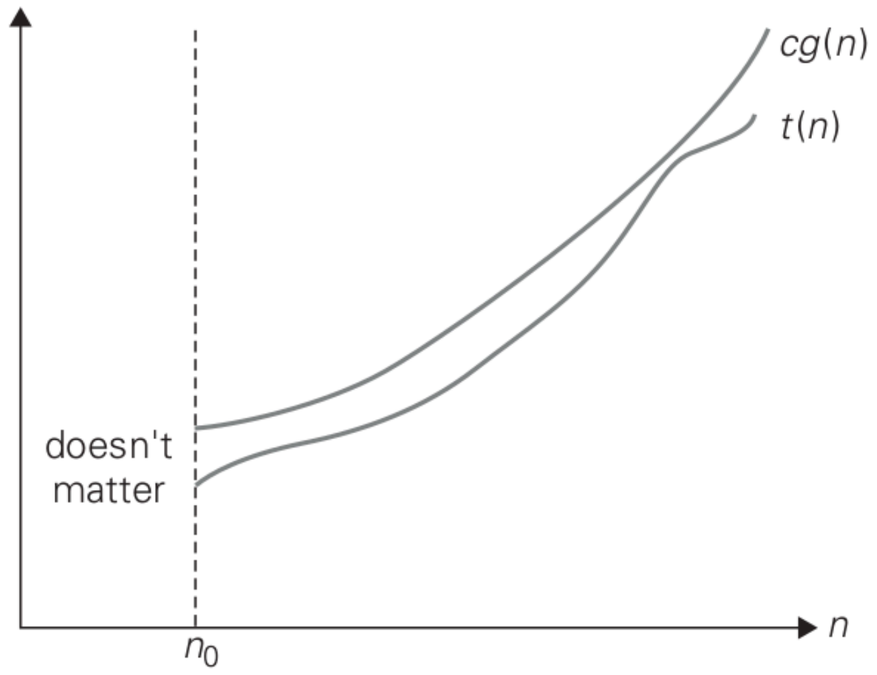


FIGURE 2.1 Big-oh notation: $t(n) \in O(g(n))$.

$$n \in O(n^2), \qquad 100n + 5 \in O(n^2), \qquad \frac{1}{2}n(n-1) \in O(n^2).$$

$$n^3 \notin O(n^2), \qquad 0.00001n^3 \notin O(n^2), \qquad n^4 + n + 1 \notin O(n^2)$$

Ω-notation

DEFINITION A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n, i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$t(n) \geq cg(n) \quad \text{for all } n \geq n_0.$$

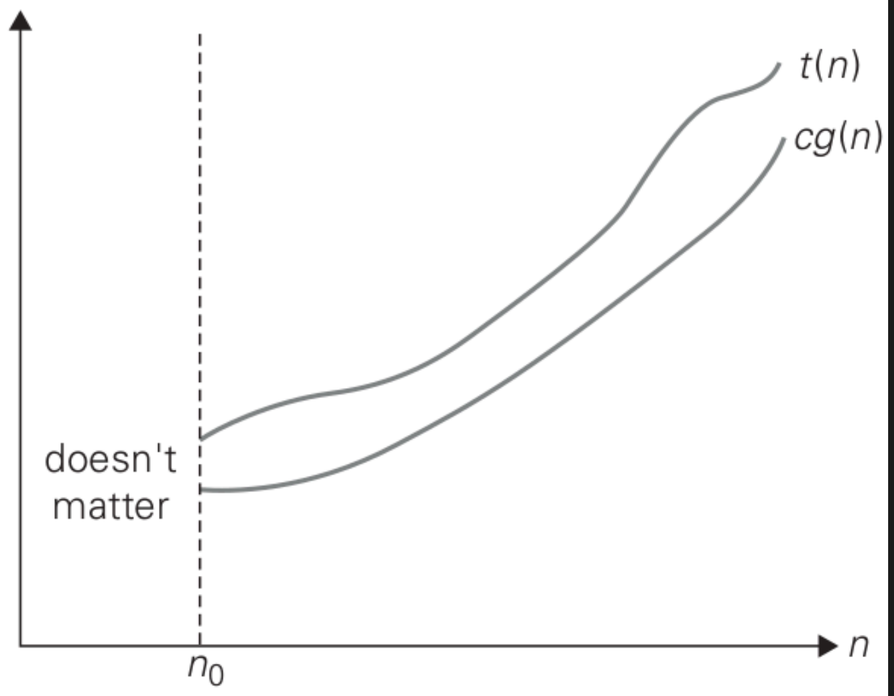


FIGURE 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$.

$$n^3 \in \Omega(n^2), \qquad \frac{1}{2}n(n-1) \in \Omega(n^2), \qquad \text{but } 100n + 5 \notin \Omega(n^2)$$

Useful Property of Asymptotic Notation

THEOREM If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\}).$$

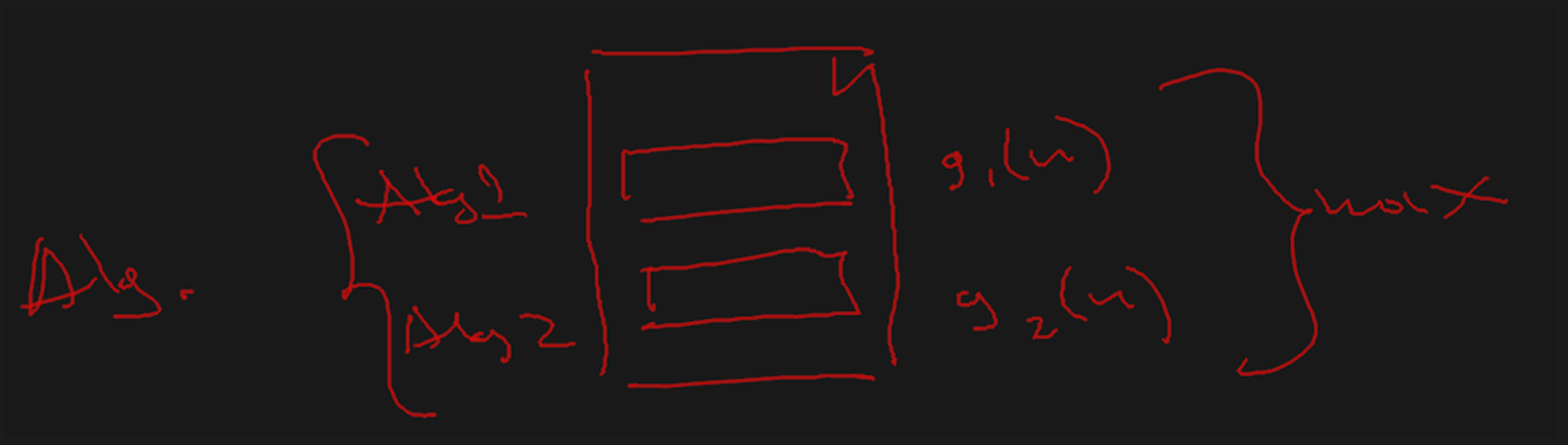


TABLE 2.2 Basic asymptotic efficiency classes

Class	Name	Comments
1	*constant*	Short of best-case efficiencies, very few reasonable examples can be given since an algorithm's running time typically goes to infinity when its input size grows infinitely large.
$\log n$	*logarithmic*	Typically, a result of cutting a problem's size by a constant factor on each iteration of the algorithm (see Section 4.4). Note that a logarithmic algorithm cannot take into account all its input or even a fixed fraction of it: any algorithm that does so will have at least linear running time.
n	*linear*	Algorithms that scan a list of size n (e.g., sequential search) belong to this class.
$n \log n$	*linearithmic*	Many divide-and-conquer algorithms (see Chapter 5), including mergesort and quicksort in the average case, fall into this category.
n^2	*quadratic*	Typically, characterizes efficiency of algorithms with two embedded loops (see the next section). Elementary sorting algorithms and certain operations on $n \times n$ matrices are standard examples.
n^3	*cubic*	Typically, characterizes efficiency of algorithms with three embedded loops (see the next section). Several nontrivial algorithms from linear algebra fall into this class.
2^n	*exponential*	Typical for algorithms that generate all subsets of an n-element set. Often, the term "exponential" is used in a broader sense to include this and larger orders of growth as well.
$n!$	*factorial*	Typical for algorithms that generate all permutations of an n-element set.

YACHAY TECH UNIVERSITY

//01

Mathematical Analysis

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

ALGORITHM *MaxElement*$(A[0..n-1])$

```
//Determines the value of the largest element in a given array
//Input: An array A[0..n − 1] of real numbers
//Output: The value of the largest element in A
maxval ← A[0]
for i ← 1 to n − 1 do
    if A[i] > maxval
        maxval ← A[i]
return maxval
```

1. Mathematical Analysis
2. Iterative algorithm → Σ
3. Solve
4. Simplify

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \qquad \sum_{i=1}^{n} 1 = n$$

$$\sum_{i=1}^{n-1} 1 = n-1 = T(n)$$

$$T(n) = n - \cancel{1}$$

$$T(n) = n$$

$$T(n) \in O(n)$$

ALGORITHM *UniqueElements*$(A[0..n-1])$

```
//Determines whether all the elements in a given array are distinct
//Input: An array A[0..n − 1]
//Output: Returns "true" if all the elements in A are distinct
//        and "false" otherwise
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i] = A[j] return false
return true
```

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \Big[\underbrace{(n-1)}_{u} - \underbrace{(i+1)}_{l} + \underbrace{1}_{1}\Big]$$

$$\sum_{i=0}^{n-2} n - 1 - i$$

$$\sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$\star$$

$$A = \boxed{1\ 1\ 1\ 1\ 1\ \cdots\ 1\ 1\ 1\ 1}$$

$$\boxed{\sum_{i=l}^{u} 1 = u - l + 1}$$

$$\star\ (n-1) \sum_{i=0}^{n-2} 1 - \sum_{i=0}^{n-2} i$$

$$(n-1)\left[(n-2) - 0 + 1\right] - \left[\frac{(n-2)\left[(n-2)+1\right]}{2}\right]$$

$$(n-1)(n-1) - \frac{(n-2)(n-1)}{2}$$

$$(n-1)^2 - \frac{(n-2)(n-1)}{2}$$

$$\frac{(n-1)n}{2} = \frac{n^2}{2} - \frac{n}{2} \Rightarrow O(n^2)$$

ALGORITHM $F(n)$

//Computes $n!$ recursively
//Input: A nonnegative integer n
//Output: The value of $n!$
if $n = 0$ **return** 1
else return $F(n-1) * n$

B.O.

$n! = 1 \cdot 2 \cdot 3 \ldots (n-1) \cdot n$

$F(n) = F(n-1) \cdot n$

$M(n) = M(n-1) + 1$

if $n = n-1 = M(n-1-1+1) + 1$

$= M(n-2) + 2$

$$\begin{cases} M(n) = M(n-1) + 1 \\ M(0) = 0 \end{cases}$$

$M(n) = M(n-1) + 1$

$= M(n-2) + 2$

$= M(n-3) + 3$

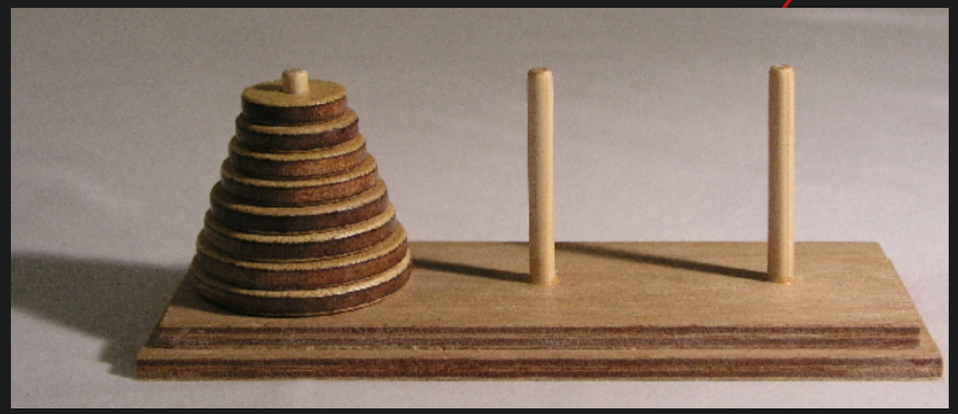
$M(n) = M(n-i) + i$

I.C. $i = n$

$\cancel{M(n-n)}^{0} + n$

$n! \in O(n)$ //

Hanoi Towers



$\boxed{61}$

$M(1) = 1$

$M(2) = 2M(2-1) + 1 = 2 \cdot 1 + 1 = 3$ ✓

$M(3) = 2M(3-1) + 1 = 2 \cdot 3 + 1 = 7$ ✓

$M(4) = 2M(4-1) + 1 = 2 \cdot 7 + 1 = 15$

$M(5) = 2M(5-1) + 1 = 2 \cdot 15 + 1 = 31$

$n \equiv$ input size

$M(n)$ = move of disk

$M(n) = M(n-1) + 1 + M(n-1)$

$M(n) = 2M(n-1) + 1$

Guess & Confirm → M.I.

$M(n) = 2^n - 1$

$M(n) \in O(2^n)$

$2^{61} - 1 \approx 2.305 \times 10^{18}$

Q & A

FOR QUESTIONS OR COMMENTS

EMAIL ADDRESS ipineda@yachaytech.edu.ec